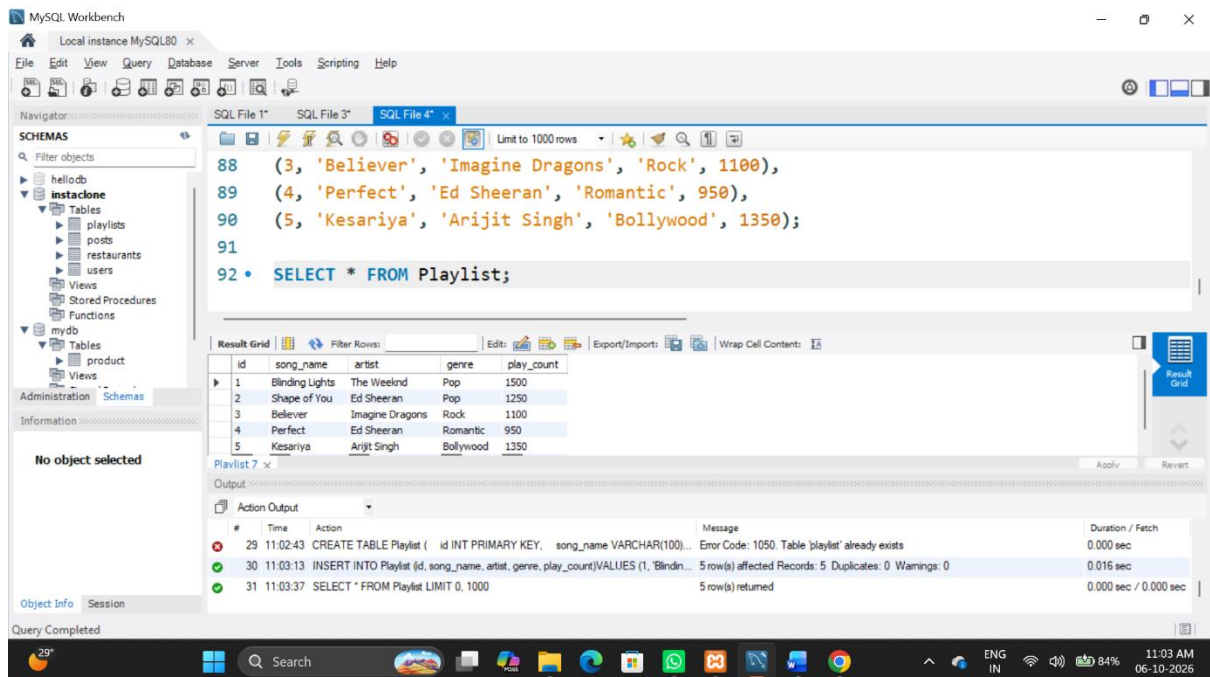


Session – 4

Q1. Create a table called Playlist with columns: id, song_name, artist, genre, and play_count. Insert at least 5 sample records representing your favorite songs from Spotify.



The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
88 (3, 'Believer', 'Imagine Dragons', 'Rock', 1100),
89 (4, 'Perfect', 'Ed Sheeran', 'Romantic', 950),
90 (5, 'Kesariya', 'Arijit Singh', 'Bollywood', 1350);
91
92 • SELECT * FROM Playlist;
```

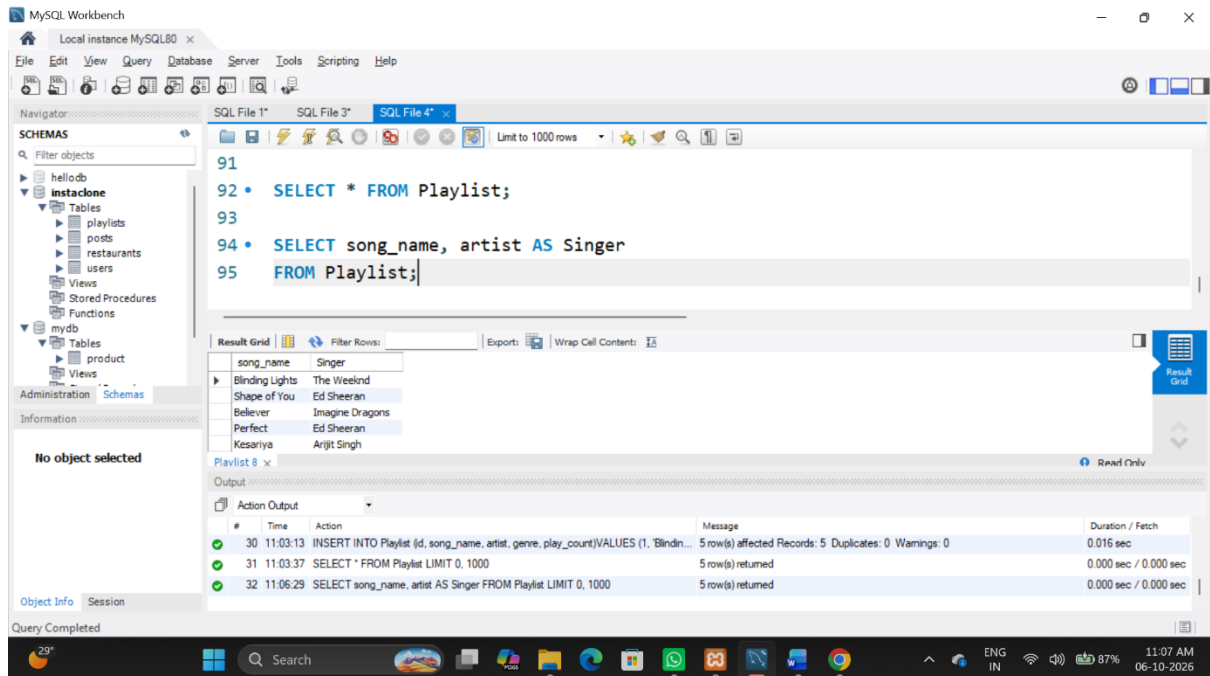
The Result Grid displays the following data:

	id	song_name	artist	genre	play_count
1	1	Blinding Lights	The Weeknd	Pop	1500
2	2	Shape of You	Ed Sheeran	Pop	1250
3	3	Believer	Imagine Dragons	Rock	1100
4	4	Perfect	Ed Sheeran	Romantic	950
5	5	Kesariya	Arijit Singh	Bollywood	1350

The Action Output pane shows the following messages:

#	Time	Action	Message	Duration / Fetch
29	11:02:43	CREATE TABLE Playlist (id INT PRIMARY KEY, song_name VARCHAR(100)...	Error Code: 1050. Table 'playlist' already exists	0.000 sec
30	11:03:13	INSERT INTO Playlist (id, song_name, artist, genre, play_count) VALUES (1, Blidin...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0	0.016 sec
31	11:03:37	SELECT * FROM Playlist LIMIT 0, 1000	5 row(s) returned	0.000 sec / 0.000 sec

Q2. Write a SELECT query to display only the song_name and artist from the Playlist table, but show the artist column as 'Singer' using an alias.



The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
91
92 • SELECT * FROM Playlist;
93
94 • SELECT song_name, artist AS Singer
95 FROM Playlist;
```

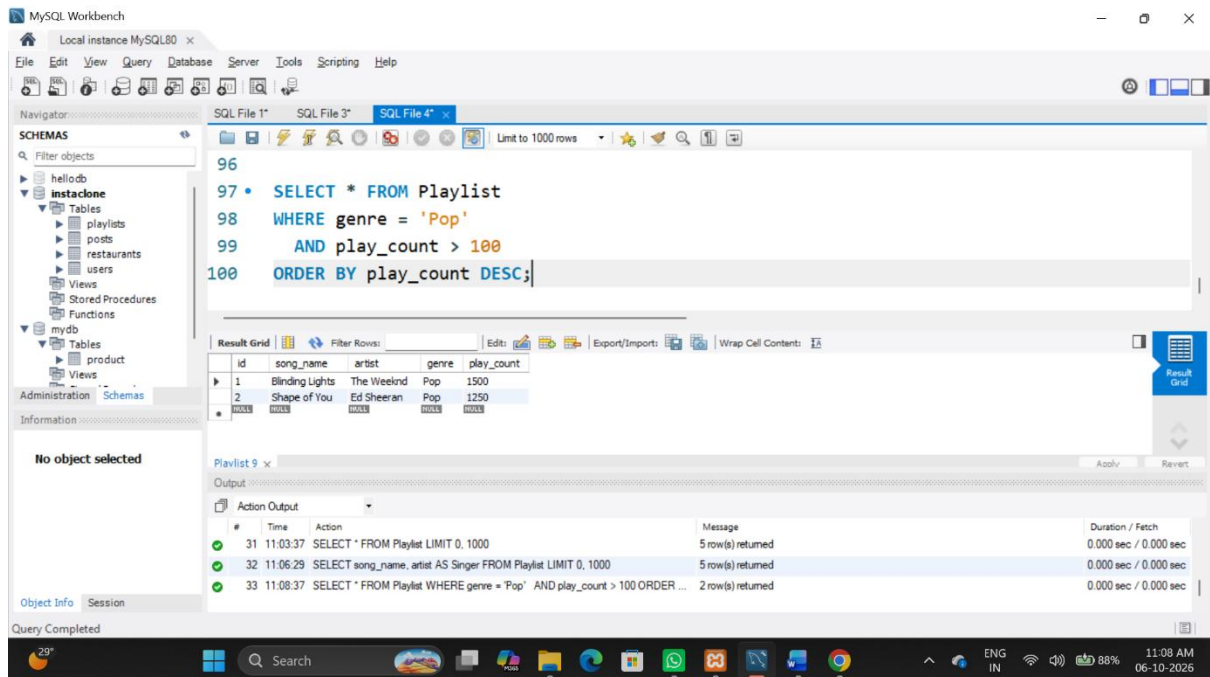
The Results window displays the output of the query. The first table shows the results of the first query (SELECT * FROM Playlist), and the second table shows the results of the second query (SELECT song_name, artist AS Singer FROM Playlist).

song_name	Singer
Blinding Lights	The Weeknd
Shape of You	Ed Sheeran
Believer	Imagine Dragons
Perfect	Ed Sheeran
Kesariya	Arijit Singh

The Output window shows the execution log:

#	Time	Action	Message	Duration / Fetch
30	11:03:13	INSERT INTO Playlist (id, song_name, artist, genre, play_count) VALUES (1, 'Blinding...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0	0.016 sec
31	11:03:37	SELECT * FROM Playlist LIMIT 0, 1000	5 row(s) returned	0.000 sec / 0.000 sec
32	11:06:29	SELECT song_name, artist AS Singer FROM Playlist LIMIT 0, 1000	5 row(s) returned	0.000 sec / 0.000 sec

Q3. Write a SELECT query to find all songs in the Playlist table where the genre is 'Pop' and play_count is greater than 100. Sort the results by play_count in descending order.



The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
96
97 * SELECT * FROM Playlist
98 WHERE genre = 'Pop'
99 AND play_count > 100
100 ORDER BY play_count DESC;
```

The Results window displays the following data:

	id	song_name	artist	genre	play_count
▶	1	Blinding Lights	The Weeknd	Pop	1500
▶	2	Shape of You	Ed Sheeran	Pop	1250

The Output window shows the execution log:

#	Time	Action	Message	Duration / Fetch
31	11:03:37	SELECT * FROM Playlist LIMIT 0, 1000	5 row(s) returned	0.000 sec / 0.000 sec
32	11:06:29	SELECT song_name, artist AS Singer FROM Playlist LIMIT 0, 1000	5 row(s) returned	0.000 sec / 0.000 sec
33	11:08:37	SELECT * FROM Playlist WHERE genre = 'Pop' AND play_count > 100 ORDER ...	2 row(s) returned	0.000 sec / 0.000 sec

Q4. Use the COUNT aggregate function to find out how many songs in your Playlist table belong to the genre 'Hip-Hop'.

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
100 ORDER BY play_count DESC;
101
102 • SELECT COUNT(*) AS hip_hop_songs
103 FROM Playlist
104 WHERE genre = 'Hip-Hop';
```

The Results window shows the output of the query:

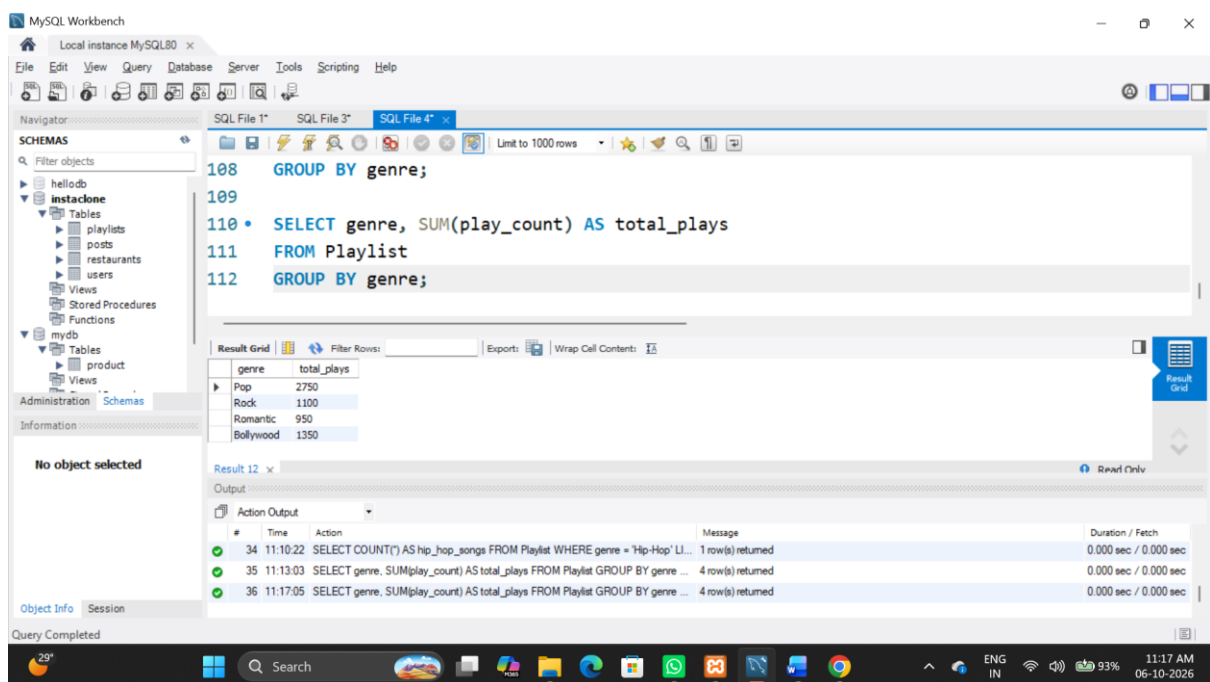
hip_hop_songs
0

The Action Output window shows the execution log:

#	Time	Action	Message	Duration / Fetch
32	11:06:29	SELECT song_name, artist AS Singer FROM Playlist LIMIT 0, 1000	5 row(s) returned	0.000 sec / 0.000 sec
33	11:08:37	SELECT * FROM Playlist WHERE genre = 'Pop' AND play_count > 100 ORDER ...	2 row(s) returned	0.000 sec / 0.000 sec
34	11:10:22	SELECT COUNT(*) AS hip_hop_songs FROM Playlist WHERE genre = 'Hip-Hop' LI...	1 row(s) returned	0.000 sec / 0.000 sec

Q5. Write a SELECT query that shows the total number of plays (SUM of play_count) for each genre in your Playlist table, grouped by genre.

Hint: Use GROUP BY with the SUM function to aggregate play counts by genre.



The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
108 GROUP BY genre;
109
110 • SELECT genre, SUM(play_count) AS total_plays
111 FROM Playlist
112 GROUP BY genre;
```

The Results window displays the output of the query as a table:

genre	total_plays
Pop	2750
Rock	1100
Romantic	950
Bollywood	1350

The Action Output window shows the execution log:

#	Time	Action	Message	Duration / Fetch
34	11:10:22	SELECT COUNT(*) AS hip_hop_songs FROM Playlist WHERE genre = 'Hip-Hop' LI...	1 row(s) returned	0.000 sec / 0.000 sec
35	11:13:03	SELECT genre, SUM(play_count) AS total_plays FROM Playlist GROUP BY genre ...	4 row(s) returned	0.000 sec / 0.000 sec
36	11:17:05	SELECT genre, SUM(play_count) AS total_plays FROM Playlist GROUP BY genre ...	4 row(s) returned	0.000 sec / 0.000 sec